

Group Cyan

WeVoteLive

Software Design Document

SE 3313B:
Section 001

Date: 04/10/2025

TABLE OF CONTENTS

1.0 INTRODUCTION	3
<i>Purpose</i>	<i>3</i>
<i>Scope</i>	<i>4</i>
2.0 SYSTEM OVERVIEW	4
3.0 SYSTEM ARCHITECTURE	5
<i>Architectural Design</i>	<i>5</i>
<i>Decomposition Description</i>	<i>7</i>
<i>Design Rationale</i>	<i>8</i>
4.0 DATA DESIGN	8
<i>Data Description</i>	<i>8</i>
<i>Data Dictionary</i>	<i>9</i>
5.0 COMPONENT DESIGN	12
6.0 HUMAN INTERFACE DESIGN	18
6.1 <i>Overview of User Interface</i>	18
6.2 <i>Screen Images</i>	19
6.3 <i>Screen Objects and Actions</i>	25
7.0 REQUIREMENTS MATRIX	27
8.0 STATEMENT OF CONTRIBUTION	27

1.0 INTRODUCTION

This document details the architecture and system design for WeVoteLive—a fast, engaging, and ephemeral social polling application built exclusively for private interactions among friends. In WeVoteLive, users do not create accounts or use persistent user names; instead, the entire experience is transient. A unique room code is generated each time a host creates a poll room, and only users with that code can join the conversation. This design choice ensures complete privacy and simplicity, focusing on fun, real-time social interaction. The software is designed to support lightweight, private polling sessions where a host can initiate multiple polls, manage when results are revealed, and allow participants to both vote and predict outcomes. The exclusivity provided by private rooms eliminates any publicly accessible or community-based polls, keeping every session contained, personal, and ephemeral.

Purpose

This Software Design Document (SWDD) serves as the definitive guide for the design, implementation, and evaluation of the WeVoteLive application. The purpose of this document is to:

- **Define the Architecture:** Establish the system architecture for an entirely private and ephemeral social polling application that operates without persistent user accounts or profiles.
- **Elucidate Core Functionalities:** Detail the core functions—including room code-based access, poll creation by a host, vote casting and outcome prediction by participants, and the controlled, on-demand revelation of poll results.
- **Specify System Constraints and Roles:** Clearly articulate that the application supports only private polls (with no public or community polls) through auto-generated room codes. This document delineates user roles: a host who creates rooms and manages polls, and participants who can vote and predict outcomes without capabilities to create or reveal polls.
- **Identify the Intended Audience:** Provide both technical and non-technical stakeholders—such as developers, system architects, product managers, and integration partners—with a comprehensive understanding of the application’s design and interactions between its components.
- **Facilitate Future Enhancements:** Offer a structured framework that both guides current implementation and allows for future extensions, such as enhancing result analytics or adding optional interactive features, while preserving the application’s core principles of simplicity, privacy, and real-time social engagement.

By removing traditional social media elements like persistent user profiles and public/community poll options, WeVoteLive is uniquely positioned to deliver a secure, fun, and streamlined experience based solely on invitation via a private room code.

Scope

WeVoteLive is engineered to provide a polling platform that emphasizes privacy, ease-of-use, and immediate social interaction. The scope of this project is defined by the following elements:

- **Private Polling Environment:**
 - Every poll room is created by a host and secured by a unique, auto-generated room code. Only users with this code can join and participate, ensuring all sessions remain private and secure.
- **Ephemeral and Lightweight Design:**
 - The application does not retain user profiles or long-term poll history. Each session is self-contained and exists only for the duration of the polling event, enhancing privacy and reducing data management overhead.
 - By eliminating the need for account creation and complex user management, the focus is placed on the simplicity of starting, managing, and participating in polls using only a room code.
- **Core Functionalities and User Roles:**
 - Hosts can create multiple polls within a room and decide when to reveal the results as well as delete the poll if they feel the need to do so. This control ensures that the flow of the poll remains in the hands of the creator, contributing to a dynamic and engaging user experience.
 - Participants cast votes and make predictions about poll outcomes. Upon the host's decision to reveal a poll, the system displays the results and indicates whether the participant's vote and prediction were correct.
 - In keeping with the design goals, hosts are solely responsible for poll management, they cannot vote or predict, while participants are dedicated to engaging with the poll content, without privileges to create or reveal poll results.
- **Technical Infrastructure:**
 - The application uses a C++-based backend to manage WebSockets controlling all requests, poll data, and vote processing, with JSON in-memory storage for transient session data.

The overall objective of WeVoteLive is to deliver a highly secure and engaging platform for private polling, eliminating unnecessary complexities while enhancing the immediacy and enjoyment of group interactions.

2.0 SYSTEM OVERVIEW

WeVoteLive is designed as a room-based social polling application that emphasizes real-time, ephemeral interactions between a host and a set of participants. The system is built with two principal layers:

- **Backend (Server) Layer:**

Implemented in C++ using the ixWebSocket library, this layer serves as the backbone of the application. It is responsible for establishing and handling secure WebSocket connections, managing room and poll data in memory (using JSON for serialization),

processing votes and predictions, and coordinating the lifecycle of polling sessions. Key functionalities include:

- **Room and Poll Management:** Automatically generating unique room and poll IDs, handling creation and deletion of polls, and managing ephemeral session data.
- **Vote Processing:** Accepting real-time vote and prediction updates from participants, updating poll statistics, and enforcing role-based permissions (e.g., only the host may create and reveal polls while participants only vote or predict).
- **Threaded Communication:** Using a multi-threaded approach with mutex locks and condition variables to safely distribute messages to clients, handle keepalives, and manage connection state.
- **Expiry and Cleanup:** Automatically expiring rooms once the host disconnects and notifying participants via error messages to ensure stale sessions are removed from the server.
- **Frontend (Client) Layer:**

Developed using SvelteKit, the client interface provides two distinct modes:

- **Host Interface:** Allows the host to generate a room code, create new polls, view current polls, reveal poll outcomes, and see associated vote statistics and predictions. It also provides visual cues (e.g., QR codes and transition animations) that reinforce the ephemeral, interactive nature of the application.
- **Participant Interface:** Enables participants to join a room using a short room code, cast their votes and predictions, and view live updates on poll outcomes once revealed by the host.

The client layer communicates with the server via a WebSocket connection using a dedicated Communication module (written in TypeScript) that manages reconnection logic, error handling, and state transitions.

Together, these components form a system optimized for real-time social polling with a strong emphasis on privacy and simplicity. The architecture ensures that user interactions are fast and immediate while session data remains transient and self-contained.

3.0 SYSTEM ARCHITECTURE

Architectural Design

1. WebSocket Communication Subsystem (Backend):

- **Connection Handling:** Uses the `ixWebSocket` library to accept and manage WebSocket connections from both hosts and participants.
- **Message Dispatching:** Implements a global message queue and a dispatcher thread that orders messages by priority and distributes updates based on room membership.
- **Keepalive and Error Handling:** Periodically sends “keepalive” pings, monitors connection state, and handles errors by building error messages that are pushed to the client.

2. Poll and Room Management Subsystem (Backend):

- **Session Management:** Generates unique room codes and poll IDs while ensuring no collisions occur. Supports ephemeral session data that automatically expires when a room is abandoned.
- **Vote and Prediction Processing:** Handles vote updates from participants and updates poll statistics accordingly. Enforces role restrictions so that only hosts can create or reveal polls.
- **Data Persistence:** Uses in-memory data structures (with synchronization via mutexes) to store poll information, participant votes, and room metadata temporarily in JSON format.

3. Thread Management and Dispatch Subsystem (Backend):

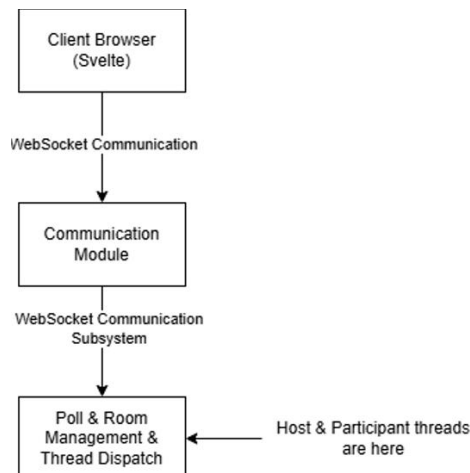
- **Dedicated Threads per Connection:** A separate thread for each connection (host or participant) manages personalized message queues, ensuring low-latency updates.
- **Global Dispatcher:** Aggregates and prioritizes outgoing messages for distribution to all clients in a room, maintaining consistency and synchronization across multiple threads.

4. Client Interface and Communication Subsystem (Frontend):

- **User Experience (SvelteKit Client):** Provides an engaging and responsive UI for hosts and participants. The interface supports creating rooms, polling interactions, and dynamic visual feedback including QR codes and animations.
- **Communication Module:** A dedicated TypeScript class wraps the native WebSocket and handles connection establishment, message serialization/deserialization, reconnections, and error propagation to the user.
- **UI Components:** Modular Svelte components (such as PollBubble, PollHeader, PollVoteCard, etc.) are responsible for rendering individual elements of the poll and core aspects of the design system, while supplying transitions and animations to improve the overall feel and interactivity.

Collaboration Between Subsystems

A simplified diagram of the high-level interactions between these subsystems is illustrated below:



Data Flow: When a client sends a message (such as a vote or poll-creation command), the Communication Module serializes the message and sends it over the WebSocket connection to the server. The WebSocket Communication Subsystem then routes the message to the appropriate handler (based on the message type). Meanwhile, updates to poll data trigger the dispatcher to send state updates back to all clients in the room.

Synchronization: Mutexes and condition variables ensure that concurrent access to shared data structures (polls, votes, room mappings) is safe and efficient. Dedicated threads ensure that both host and participant sessions are updated in near-real-time.

Decomposition Description

The system's functional decomposition can be summarized with the following outline:

- **Connection Lifecycle:**
 - **On Connection:** When a client connects, a shared pointer is associated with a new or existing room.
 - **Message Processing:** Upon receipt of a message, the server parses the JSON payload and dispatches the event to a corresponding handler (e.g., "host-hello", "join-hello", "vote", "create-poll").
 - **Threaded Message Dispatch:** Each connection thread maintains its own message queue for sending targeted updates back to its associated client.
- **Poll Operations:**
 - **Poll Creation:** Handled exclusively by hosts, invoking the handler that generates a unique poll ID, instantiates a new Poll object, and appends it to the room's poll collection.
 - **Vote Processing:** In the "vote" handler, user selections are captured, and corresponding counters within the poll object are incremented. Voting updates are then broadcast to all clients within that room.
 - **Poll Reveal and Delete:** Host-initiated actions mark polls as "revealed" or remove them from the system and trigger notifications to update the UI of all active clients.
- **Room Expiry Mechanism:**
 - **Expiry Scheduling:** When the host disconnects, the room is flagged for expiry (with a defined timeout period).
 - **Cleanup Thread:** A dedicated thread continuously checks for expired rooms and purges associated data (polls, votes, participant mappings) from memory. Clients in expired rooms are notified accordingly.
- **UI and Client-Side Interactions:**
 - **Component-Based Rendering:** The Svelte frontend is organized into components that encapsulate distinct parts of the poll interface (voting, predictions, results, etc.).
 - **Animation and Transitions:** Custom transitions (e.g., vertical expansion, scaling, fading) are applied to improve the user experience.
 - **Communication Management:** The client's Communication class handles the entire lifecycle of the WebSocket connection, ensuring that messages are sent and received correctly and that state transitions (connecting, error, normal operation) are reflected in the UI.

Design Rationale

The chosen architecture is driven by the key project goals of privacy, real-time interaction, and simplicity:

- **Real-Time and Ephemeral Interaction:**
The use of WebSocket technology (via ixWebSocket) supports persistent, low-latency connections essential for real-time polling interactions. With an emphasis on ephemeral sessions, the system avoids persistent state storage, reinforcing privacy and minimizing server maintenance.
- **Simplicity and Modularity:**
By dividing responsibilities among clearly defined subsystems (connection handling, poll processing, UI rendering), the design remains modular and easily maintainable. Each module's well-defined interfaces facilitate debugging and future enhancements.
- **Role-Based Data Management:**
The differentiation between host and participant roles simplifies the logic; hosts control poll creation and result revelation while participants are limited to voting and prediction. This separation reduces the risk of unauthorized actions and improves overall reliability.
- **Threaded Concurrency with Data Safety:**
The server employs multiple threads for handling client communications without sacrificing data consistency. The use of mutexes and condition variables ensures that state updates (votes, poll creation, room expiry) occur safely even under high concurrency.
- **Scalability and Extendibility:**
The type-safe design of the client Communication module combined with in-memory poll handling establishes a baseline that can be scaled horizontally if required. Additional features (such as advanced analytics or external integrations) can be built atop this framework without compromising the core focus on private, ephemeral interactions.

In summary, the architecture reflects careful considerations of performance, security, and ease of use. It meets the project's requirements by ensuring that all interactions remain private, real-time, and focused solely on the fun dynamics of live polling.

4.0 DATA DESIGN

Data Description

The WeVoteLive system focuses on an ephemeral, real-time polling experience. The design leverages in-memory data structures rather than a traditional persistent database. This is intentional given the transient nature of the application, where sessions (rooms) and poll data are maintained only for the duration of the live interaction. Key aspects include:

- **Ephemeral In-Memory Storage:**
All polling data (rooms, polls, votes) are stored in in-memory containers (e.g., C++ STL unordered maps and vectors). This approach significantly reduces data management overhead while supporting rapid read/write operations required for real-time

performance.

- **JSON Serialization:**

Data structures are converted to and from JSON using the *nlohmann::json* library. JSON serialization is used to encapsulate the poll state, user votes, and error or system messages into a format that is easily transmitted over WebSocket connections to client-side interfaces.

- **Major Data Entities and Their Relationships:**

- **Room Management:**

- Each polling session corresponds to a unique room identified by a unique room code. The room data includes a mapping of the host and participant identities as well as pointers to the list of active polls in that room.

- **Poll Entity:**

- Polls are represented by a custom Poll structure which encapsulates the question, description, available options, vote counts, prediction counts, and a flag indicating whether the poll results are revealed. Polls are stored inside a vector that is associated with the room ID.

- **User Votes:**

- User votes and predictions for each poll are stored in a three-tiered data structure. The hierarchy associates a room ID with participant identities and, further, with poll IDs. For each poll, a UserVote structure captures the participant's vote and prediction choices (represented as booleans indicating whether option A was selected for vote or prediction).

- **Message Passing:**

- Outbound and inbound communication is handled via message objects. A Message structure contains the JSON payload to be sent, a reference to the associated WebSocket, a priority level (to manage ordering and responsiveness), and the related room code.

- **Thread and Connection State:**

- To handle concurrency, objects of type ConnectionThreadInfo are created for each client connection (whether host or participant). These objects maintain personal queues for messages, thread handles for asynchronous communication, and connection metadata (identity, type, roomCode).

- **Synchronization and Expiry:**

Several mutexes and condition variables are used to ensure thread-safe access to the shared data. Additionally, an expiry mechanism tracks inactive rooms so that they are automatically purged after a defined period (e.g., 1 minute after the host disconnects), ensuring efficient cleanup of ephemeral data.

No external or persistent databases are used since the transient nature of these interactions renders an in-memory model ideal for both performance and security.

Data Dictionary

Below is an alphabetical list of the major system entities, data types, functions, and their purposes as used in WeVoteLive.

Data Entities

- **ConnectionThreadInfo**
 - Type: Structure / Object
 - Attributes:
 - type (string): The role of the connection (e.g., “host” or “participant”).
 - identity (string): A unique identifier for the connected client.
 - roomCode (string): The room code with which the client is associated.
 - socket (shared_ptr::WebSocket): A pointer to the WebSocket connection.
 - threadHandle (thread): The dedicated thread handling this connection’s message queue.
 - personalQueue (deque<Message>): Queue for messages directed to this connection.
 - personalCond (condition_variable): Condition variable for synchronizing queue operations.
 - personalMutex (mutex): Mutex to protect concurrent access to the personal queue.
 - running (bool): Flag that indicates if the connection is active.
- **Message**
 - Type: Structure / Object
 - Attributes:
 - payload (json): JSON object containing the message content.
 - socket (shared_ptr::WebSocket): The WebSocket over which the message is sent.
 - priority (int): Priority level of the message (lower values indicate higher priority).
 - roomCode (string): The associated room code used to determine recipients.
- **Poll**
 - Type: Structure / Object
 - Attributes:
 - id (string): The unique identifier for the poll.
 - question (string): The text of the poll question.
 - description (string): Details or supplementary information regarding the poll.
 - optionA (string): Description for option A.
 - optionB (string): Description for option B.
 - predictionsA (int): The count of predictions for option A.
 - predictionsB (int): The count of predictions for option B.
 - votesA (int): The number of votes for option A.
 - votesB (int): The number of votes for option B.
 - revealed (bool): Flag indicating whether the poll’s results are revealed.
- **UserVote**
 - Type: Structure / Object
 - Attributes:
 - votedA (optional<bool>): Indicates if the vote was cast for option A.

- `predictedA` (optional<bool>): Indicates if the prediction was made in favor of option A.

Global Data Structures (Backend)

- **allPolls**
 - Type: `unordered_map<string, vector<Poll>>`
 - Description: Maps room IDs to a vector of Poll objects stored for that room.
- **roomParticipantMap**
 - Type: `unordered_map<string, unordered_map<string, string>>`
 - Description: Associates each room code with a mapping of participant identities. Includes a reserved “host” identifier.
- **allVotes**
 - Type: `unordered_map<string, unordered_map<string, unordered_map<string, UserVote>>>`
 - Description: Maintains a hierarchical mapping from room ID to participant identity and poll ID with the corresponding UserVote record.
- **expiringRooms**
 - Type: `unordered_map<string, chrono::steady_clock::time_point>`
 - Description: Tracks rooms scheduled for expiry based on host disconnection times.
- **globalMessageQueue**
 - Type: `deque<Message>`
 - Description: A shared queue used to store and dispatch messages from the server to the appropriate client connections.

Utility Functions (with Parameters)

- **generateRandomID() → string**
 - Description: Generates a 4-character random alphanumeric ID used as a component for room codes and poll IDs.
- **generateUniqueRoomID() → string**
 - Description: Uses `generateRandomID()` to create a room code ensuring no collisions exist in `roomParticipantMap`, `allPolls`, or `expiringRooms`.
- **generateUniquePollID(roomCode: string) → string**
 - Description: Generates a unique poll ID within the specified room by checking against the current polls in `allPolls`.
- **generateUniqueSessionID() → string**
 - Description: Creates a longer, unique session identifier used to differentiate between client sessions.
- **buildErrorMessage(ws: shared_ptr::WebSocket, error: string, retry: bool) → Message**
 - Description: Constructs an error message (as a JSON payload) to be sent back to clients in cases of unexpected issues.

JSON Converters

- **to_json(json &j, const Poll &p)**
 - Description: Serializes a Poll object into a JSON representation.
- **to_json(json &j, const UserVote &u)**
 - Description: Serializes a UserVote object into JSON form.

5.0 COMPONENT DESIGN

This section details the behavior of the individual components in WeVoteLive. Each component is described in functional (procedural) pseudocode that outlines its algorithm and local data management. The overall design follows the decomposition described in Section 3.2, with components handling connection management, poll operations, message dispatching, and room expiry.

Connection and Thread Management

Component: ConnectionThreadInfo

Responsibilities:

- Store metadata about each client session (host or participant).
- Manage a personal message queue and synchronization mechanisms (mutex and condition variable).
- Maintain the running status for the connection thread.

Pseudocode:

```
Structure ConnectionThreadInfo:
  Attributes:
    type: string           // "host" or "participant"
    identity: string       // Client unique identity
    roomCode: string      // Associated room code
    socket: WebSocket      // Connection socket
    threadHandle: Thread   // Thread running for this connection
    personalQueue: deque<Message> // Queue to hold messages for this connection
    personalCond: ConditionVariable // For signaling message arrival
    personalMutex: Mutex   // Protects access to personalQueue
    running: Boolean       // True if connection is active, false otherwise

    // No explicit functions; used by thread functions defined below.
```

Component: Host and Participant Thread Functions

Each connection (host or participant) spawns a thread that continuously checks for new messages or send keepalive pings when idle.

Pseudocode for Thread Function (common pattern):

```
Function connectionThreadRun(connectionInfo: ConnectionThreadInfo)
  WHILE connectionInfo.running IS TRUE DO
    LOCK connectionInfo.personalMutex
    WAIT for personalCond with a timeout of 20 seconds OR until personalQueue is not empty
    IF connectionInfo.running IS FALSE
      BREAK
    IF no message available THEN
      UNLOCK connectionInfo.personalMutex
      TRY
        connectionInfo.socket.sendPing("keepalive")
      CATCH exception THEN
        LOG error sending ping
      END TRY
    CONTINUE
  END IF
  message ← personalQueue.popFront()
  UNLOCK connectionInfo.personalMutex
  TRY
```

```

        connectionInfo.socket.send(message.payload)
    CATCH exception THEN
        LOG error sending message
    END TRY
END WHILE
EndFunction

```

- **Local Data:**

Each thread uses its private personalQueue, personalMutex, and personalCond to handle its message dispatch in a thread-safe manner. The pattern is similar for both host and participant threads (created in createHostThread and createParticipantThread respectively).

WebSocket Message Handling

Component: onMessage() Callback Function

Responsibilities:

- Handle incoming WebSocket messages.
- Dispatch actions based on the message type by invoking registered handler functions.
- Log connection events (open, message, close, error).

Pseudocode:

```

Function onMessage(socket: WebSocket, message: WebSocketMessage)
    IF message.type IS "Message" THEN
        receivedText ← message.string
        LOG "Received message from [identity]: " + receivedText
        parsedMessage ← parseJSON(receivedText)
        IF parsedMessage does NOT contain "type" THEN
            errorMsg ← buildErrorMessage(socket, "missing-type", retry=true)
            push errorMsg to globalMessageQueue
            SIGNAL queueCond
            RETURN
        END IF
        actionType ← parsedMessage["type"]
        IF handler for actionType EXISTS THEN
            CALL handler(actionType)(parsedMessage, socket)
        ELSE
            errorMsg ← buildErrorMessage(socket, "invalid-type", retry=true)
            push errorMsg to globalMessageQueue
            SIGNAL queueCond
        END IF
    ELSE IF message.type IS "Open" THEN
        LOG "Client connected."
    ELSE IF message.type IS "Close" THEN
        CALL handleDisconnection(socket)
    ELSE IF message.type IS "Error" THEN
        LOG "WebSocket Error: " + message.errorInfo
    END IF
EndFunction

```

Local Data:

Uses global data structures (e.g., allConnectionThreads, globalMessageQueue) for lookup and dispatch.

Poll and Room Management

Component: Poll Creation Handler ("create-poll")

Responsibilities:

- Allow the host to create a poll.

- Generate a unique poll ID.
- Instantiate a new Poll object and insert it into the room's poll collection.

Pseudocode:

```

Handler "create-poll"(message, socket):
  // Extract poll details from message
  question ← message["question"]
  description ← message["description"]
  optionA ← message["optionA"]
  optionB ← message["optionB"]

  // Validate: ensure socket is associated with an active host thread
  connectionInfo ← lookupConnection(socket)
  IF connectionInfo.type != "host" THEN
    errorMsg ← buildErrorMessage(socket, "participant-poll-creation", retry=true)
    push errorMsg to globalMessageQueue
    SIGNAL queueCond
    RETURN
  END IF

  // Generate a unique poll ID for the room
  pollId ← generateUniquePollID(connectionInfo.roomCode)

  // Create Poll instance
  newPoll ← new Poll(pollId, question, description, optionA, optionB)

  // Insert poll into the room's poll list (with thread safety)
  LOCK globalStorageMutex
  allPolls[connectionInfo.roomCode].push_back(newPoll)
  UNLOCK globalStorageMutex

  // Respond to the host with updated poll state
  responseMsg ← buildMessageForClient(socket, connectionInfo.roomCode)
  set responseMsg.priority ← 0
  push responseMsg to globalMessageQueue
  SIGNAL queueCond
EndHandler

```

Component: Vote Processing Handler ("vote")

Responsibilities:

- Accept and validate votes from participants.
- Update the poll vote counts and prediction counts based on the input.
- Store individual user votes into the allVotes data structure.

Pseudocode:

```

Handler "vote"(message, socket):
  pollId ← message["for"]
  votedA ← message["votedA"]
  predictedA ← message["predictedA"]

  connectionInfo ← lookupConnection(socket)
  IF connectionInfo.type != "participant" THEN
    errorMsg ← buildErrorMessage(socket, "host-vote", retry=false)
    push errorMsg to globalMessageQueue
    SIGNAL queueCond
    RETURN
  END IF

  LOCK globalStorageMutex
  pollsList ← allPolls[connectionInfo.roomCode]

```

```

poll ← find poll in pollslst where poll.id == pollId
IF poll DOES NOT exist THEN
    errorMsg ← buildErrorMessage(socket, "invalid-poll-vote", retry=false)
    push errorMsg to globalMessageQueue
    UNLOCK globalStorageMutex
    SIGNAL queueCond
    RETURN
END IF

// Store user vote (and prediction) in the hierarchical vote mapping
allVotes[connectionInfo.roomCode][connectionInfo.identity][pollId].votedA ← votedA
allVotes[connectionInfo.roomCode][connectionInfo.identity][pollId].predictedA ← predictedA

// Update poll counters based on vote and prediction
IF votedA IS TRUE THEN
    poll.votesA ← poll.votesA + 1
ELSE
    poll.votesB ← poll.votesB + 1
END IF
IF predictedA IS TRUE THEN
    poll.predictionsA ← poll.predictionsA + 1
ELSE
    poll.predictionsB ← poll.predictionsB + 1
END IF
UNLOCK globalStorageMutex

// Notify the participant with updated poll state
responseMsg ← buildMessageForClient(socket, connectionInfo.roomCode)
set responseMsg.priority ← 3
push responseMsg to globalMessageQueue
SIGNAL queueCond
EndHandler

```

Component: Reveal Poll Handler ("reveal-poll")

Responsibilities:

- Allow the host to mark a poll as revealed.
- Locate the poll in the room's collection and set its revealed flag.
- Inform all participants in the room that the poll state has changed.

Pseudocode:

```

Handler "reveal-poll"(message, socket):
    pollId ← message["id"]

    connectionInfo ← lookupConnection(socket)
    IF connectionInfo.type != "host" THEN
        errorMsg ← buildErrorMessage(socket, "participant-reveal", retry=true)
        push errorMsg to globalMessageQueue
        SIGNAL queueCond
        RETURN
    END IF

    LOCK globalStorageMutex
    pollslst ← allPolls[connectionInfo.roomCode]
    poll ← find poll in pollslst where poll.id == pollId
    IF poll EXISTS THEN
        poll.revealed ← TRUE
    ELSE
        errorMsg ← buildErrorMessage(socket, "invalid-poll-reveal", retry=true)
        push errorMsg to globalMessageQueue
        UNLOCK globalStorageMutex
        SIGNAL queueCond
        RETURN
    END IF
    UNLOCK globalStorageMutex

```

```

// Inform all clients in the room of the updated state
responseMsg ← buildMessageForClient(socket, connectionInfo.roomCode)
set responseMsg.priority ← 1
push responseMsg to globalMessageQueue
SIGNAL queueCond
EndHandler

```

Component: Delete Poll Handler ("delete-poll")

Responsibilities:

- Allow the host to delete a poll.
- Remove the poll from the poll list and purge any associated votes.
- Ensure the clients are updated with the new state.

Pseudocode:

```

Handler "delete-poll"(message, socket):
    pollId ← message["id"]

    connectionInfo ← lookupConnection(socket)
    IF connectionInfo.type != "host" THEN
        errorMsg ← buildErrorMessage(socket, "participant-poll-delete", retry=true)
        push errorMsg to globalMessageQueue
        SIGNAL queueCond
        RETURN
    END IF

    LOCK globalStorageMutex
    pollsList ← allPolls[connectionInfo.roomCode]
    found ← FALSE
    FOR each poll in pollsList DO
        IF poll.id EQUALS pollId THEN
            REMOVE poll from pollsList
            found ← TRUE
        END IF
    END FOR

    IF found IS FALSE THEN
        errorMsg ← buildErrorMessage(socket, "invalid-poll-delete", retry=true)
        push errorMsg to globalMessageQueue
        UNLOCK globalStorageMutex
        SIGNAL queueCond
        RETURN
    END IF

    // Remove associated votes from allVotes
    FOR each participant in allVotes[connectionInfo.roomCode] DO
        REMOVE entry for pollId in participant's vote map
    END FOR
    UNLOCK globalStorageMutex

    responseMsg ← buildMessageForClient(socket, connectionInfo.roomCode)
    set responseMsg.priority ← 0
    push responseMsg to globalMessageQueue
    SIGNAL queueCond
EndHandler

```

Room Expiry and Global Dispatcher

Component: Room Expiry Thread

Responsibilities:

- Continuously check for room expiry based on scheduled expiry times.
- Purge all associated data (polls, participant mappings, votes) when a room expires.
- Notify connected clients of room closure via error messages.

Pseudocode:

```
Function createRoomExpiryThread():
  LOOP forever:
    Sleep for 1 second
    now ← current time
    expiredRooms ← empty list

    LOCK globalStorageMutex
    FOR each (room, expiryTime) in expiringRooms DO
      IF now >= expiryTime THEN
        ADD room to expiredRooms list
      END IF
    END FOR
    UNLOCK globalStorageMutex

    FOR each room in expiredRooms DO
      LOCK globalStorageMutex
      REMOVE allPolls[room], roomParticipantMap[room], and allVotes[room]
      UNLOCK globalStorageMutex

      LOCK wsQueueMutex
      FOR each connectionInfo in allConnectionThreads WHERE connectionInfo.roomCode equals room
        DO
          SEND error message "room-closed" to connectionInfo.socket
          Mark connectionInfo.running as FALSE
          SIGNAL connectionInfo.personalCond
          REMOVE connectionInfo from allConnectionThreads
        END FOR
      UNLOCK wsQueueMutex

      LOCK globalStorageMutex
      REMOVE room from expiringRooms
      UNLOCK globalStorageMutex

      LOG "Room [room] has expired and has been deleted."
    END FOR
  EndFunction
```

Component: Global Dispatcher Thread

Responsibilities:

- Maintain a global message queue.
- Sort and deliver messages to each client's personal queue based on the associated room code.
- Signal threads so that messages are sent promptly.

Pseudocode:

```
Function dispatcherThreadFunction():
  LOOP forever:
    LOCK wsQueueMutex
    WAIT on queueCond until globalMessageQueue is not empty
    SORT globalMessageQueue by message priority (lower is higher priority)
    message ← globalMessageQueue.popFront()
    UNLOCK wsQueueMutex

    IF message.payload["type"] IS "error" THEN
      SEND message.payload to message.socket immediately
      CONTINUE
```

```

END IF

LOCK wsQueueMutex
FOR each connectionInfo in allConnectionThreads DO
  IF connectionInfo.roomCode EQUALS message.roomCode THEN
    tailoredMsg ← buildMessageForClient(connectionInfo.socket, message.roomCode)
    tailoredMsg.priority ← message.priority
    LOCK connectionInfo.personalMutex
    ADD tailoredMsg to connectionInfo.personalQueue
    UNLOCK connectionInfo.personalMutex
    SIGNAL connectionInfo.personalCond
  END IF
END FOR
UNLOCK wsQueueMutex
EndFunction

```

In Summary each component of the system fulfills a particular role, from handling WebSocket connections and executing per-connection threads to processing poll events and coordinating global messaging. The pseudocode provided outlines the logical flow of key routines and demonstrates how local data is managed and synchronized across threads to maintain an efficient and responsive real-time polling platform.

This detailed component design ensures that future developers and maintainers will have a clear, algorithmic overview of how the system’s fundamental operations are implemented and how they interrelate.

6.0 HUMAN INTERFACE DESIGN

6.1 Overview of User Interface

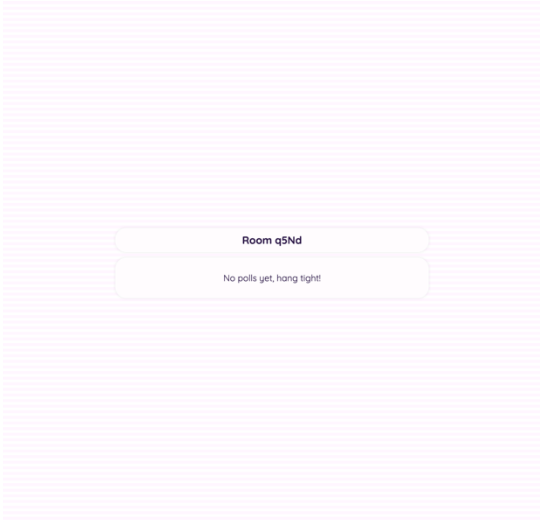
The user interface revolves around a shared list of Polls. The interface is slightly different depending on if the user is a host or participant, with each having different additional components available over the shared list of polls, each with a section showing the results (if revealed) and the predictions. As a participant, users join a room by typing a room code, scanning a QR code, or receiving a shared link. Participants can then vote on open polls they haven’t voted on, using a card displayed underneath the poll details and above the current predictions. When voting, a participant simultaneously votes for the question in the poll, as well as what they think the most popular result will be. A section under the predictions appears for users who have voted on a poll, reminding them what they voted for. If the poll is revealed, a winning vote or correct prediction has an additional badge signifying it. As a host, a “Reveal this poll!” button replaces the voting card on unrevealed polls, and a small section underneath the predictions segment hosts a button to delete a poll instead. Polls can be deleted both before and after they’ve been revealed, and doing so removes them from all participants as well. Hosts can also create new Polls using a special card on the bottom of the list containing fields for the Poll’s question and both options. Because the host’s screen is designed to be projected, hosts also have a QR code at the top-left corner of their screen, which can be scanned to join the hosted room as a participant. The only page that is not a list of Polls is the home page. If the site is navigated to without a room code, the home page is displayed, which presents the title and a form allowing users to choose if they wish to host or join a room code, which they may type.

6.2 Screen Images

No Polls



The Host



A Participant

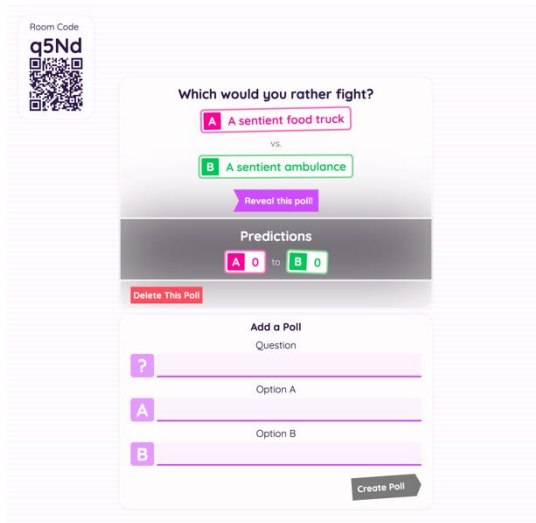
Adding a Poll



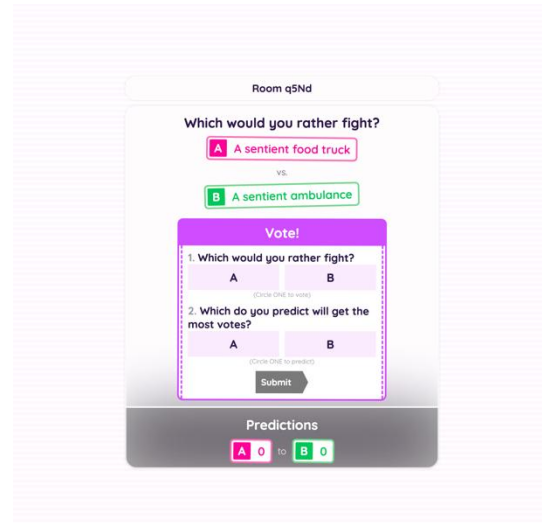
The Host

(n/a)

Viewing a new Poll

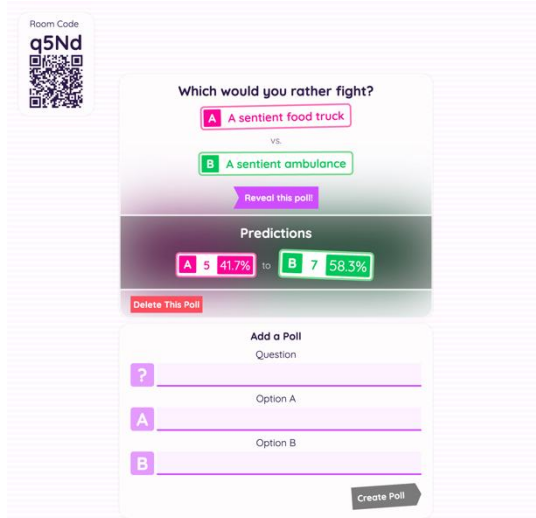


The Host

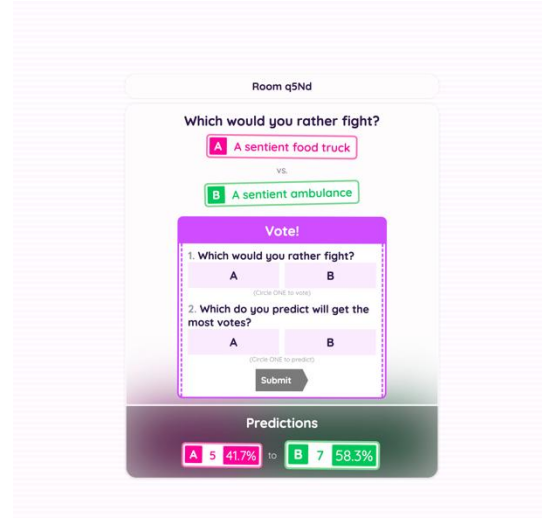


A Participant

Viewing a Poll with predictions



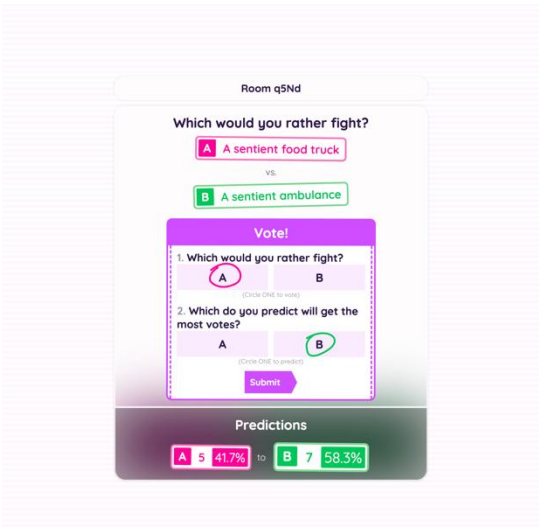
The Host



A Participant

Voting on a Poll

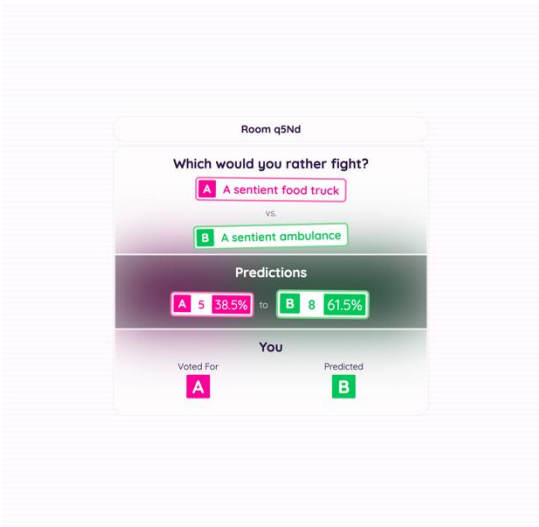
(n/a)



A Participant

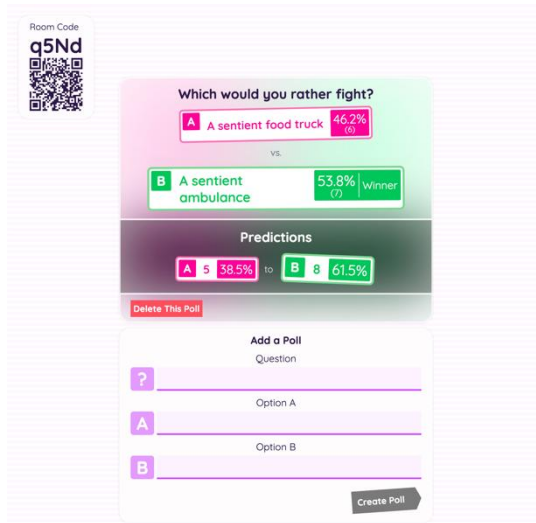
Viewing a Poll you have voted on

(n/a)

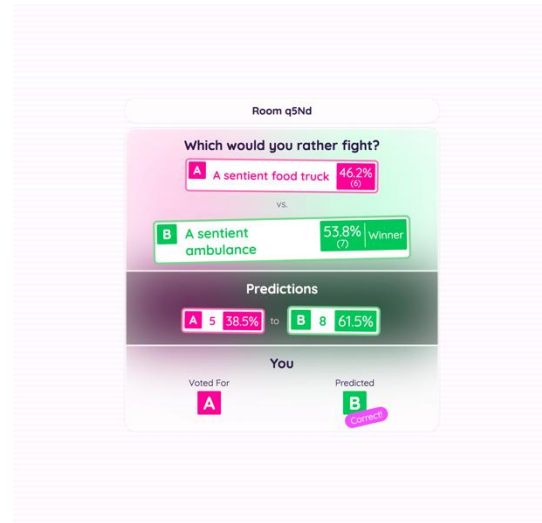


A Participant

Viewing a revealed Poll

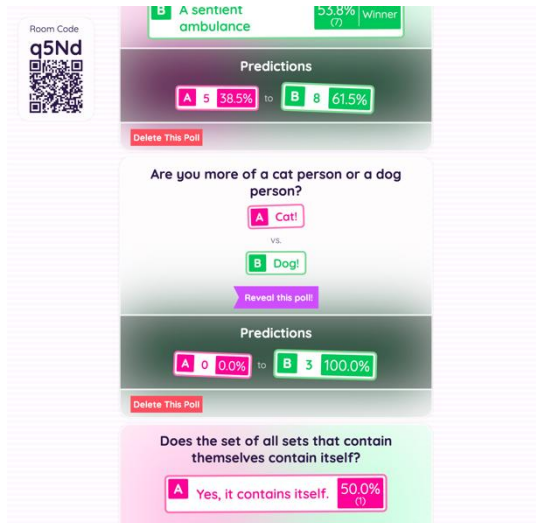


The Host

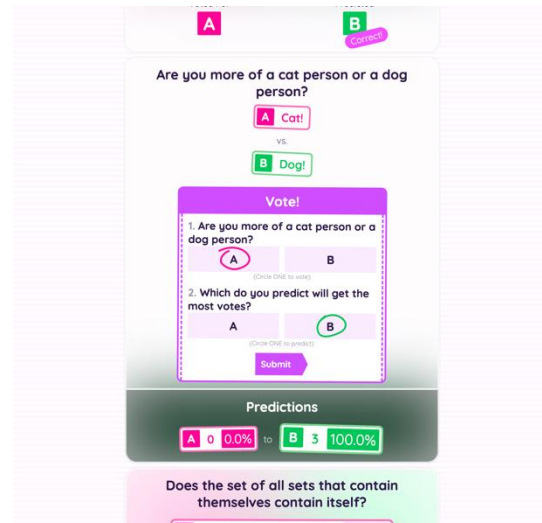


A Participant

Viewing many Polls, in various states

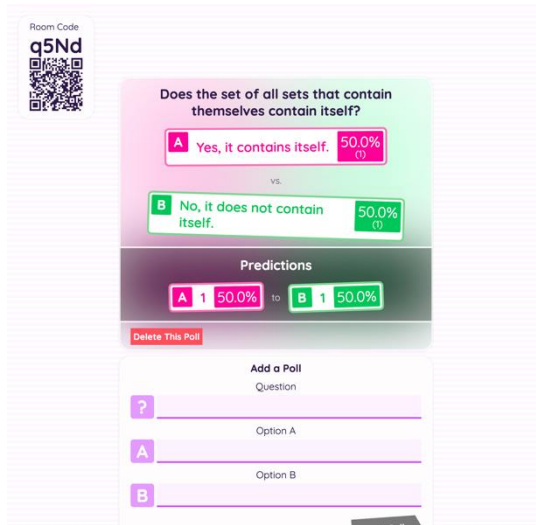


The Host

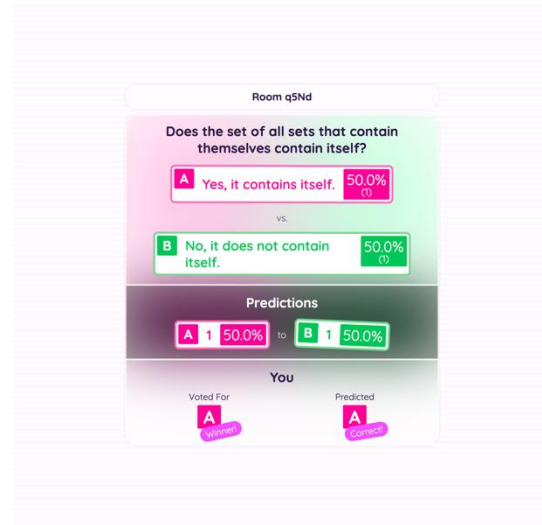


A Participant

Viewing A Tie

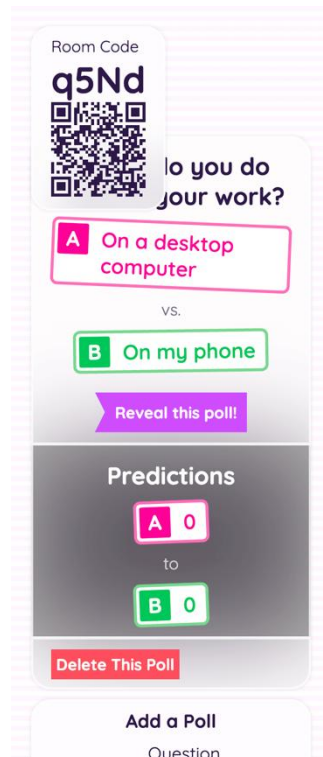


The Host

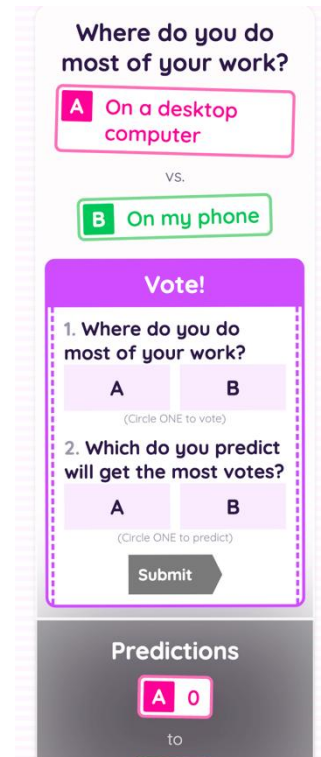


A Participant

Using the site on mobile



The Host (not recommended!)



A Participant (intended use)

The home page

WeVote **Live!**

Get started by hosting or joining a room.

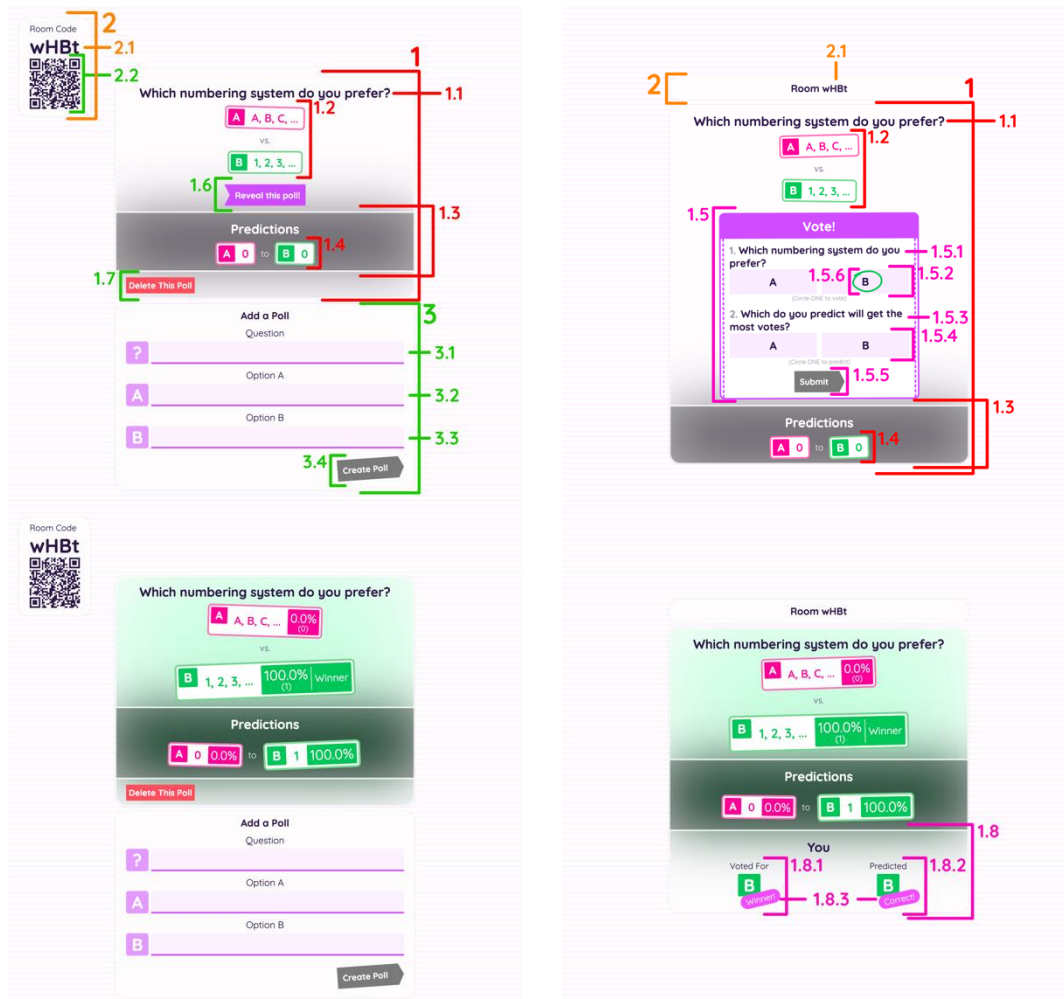
Host

Room Code

Join

Room codes are case-sensitive.

6.3 Screen Objects and Actions



Object	Name	Purpose / Action
1	Poll Card	Displays various sections of information and controls about a poll.
1.1	Poll Question	Displays the question of a Poll.
1.2	Poll Option Stickers	Display the two options in a Poll, and their results if revealed.
1.3	Predictions Section	Displays information about the received predictions of a Poll.
1.4	Prediction Stickers	Show which of the two options got how much predictions for this Poll.
1.5	Voting Card	Contains controls for a Participant to vote on a Poll.
1.5.1	Vote Question	Re-states the question, mostly for aesthetics but also for ease of use, a little.
1.5.2	Voting Radio Buttons	Allows a Participant to select which side to vote for.
1.5.3	Vote Prediction Label	Denotes the following option as for predictions, not the vote itself. Static text that does not change per poll.

1.5.4	Vote Prediction Radio Buttons	Allows a Participant to select which side to cast their prediction for.
1.5.5	Vote Submit Button	Confirms and casts a vote and prediction denoted by the appropriate radio buttons.
1.5.6	Radio Button Dynamic Circle	Highlights the selected radio button option. Dynamically randomly generated by offsetting some control points in an SVG path. Also uses the appropriate color of the selected side, for clarity.
1.6	Poll Reveal Button	Reveals the poll when clicked.
1.7	Poll Delete Button	Deletes the poll when clicked, removing it from all Participant's screens as well.
1.8	Poll You Section	Displays information to remind a Participant of what they voted on or predicted in a Poll.
1.8.1	Voted For	Displays the side the Participant voted for.
1.8.2	Predicted	Displays the side the Participant predicted would win.
1.8.3	Correct Badge	Displays "Winner!" over Voted For if that side was the most popular Poll vote and displays "Correct!" over Predicted if the prediction was correct.
2	Current Room Card	Displays the current room code, as a reminder. Displays differently on Host and Participant, being part of the scrolling list for Participants and being fixed to the top-left of the screen for Hosts, where it is more important.
2.1	Current Room	Displays the current room code. Displays larger when Hosting, as others will need to copy it. For Participants, it's just a reminder.
2.2	Room QR Code	Displays a QR code with a link to join the current room. Displayed only for the Host, to allow Participants to join by scanning a projection of the Host's screen with a mobile device.
3	Add Poll Form	A card hosting a form used by the Host to add Polls. Always displays at the bottom of the Poll list.
3.1	Question Field	Allows the Host to type in the question for the new Poll.
3.2	Option A Field	Allows the Host to type in the "A" option for the new Poll.
3.3	Option B Field	Allows the Host to type in the "B" option for the new Poll.
3.4	Create Poll Button	Submits the Add Poll Form, creating a new Poll from its fields and clearing them in the process.

7.0 REQUIREMENTS MATRIX

Requirement ID	Requirement Description	Satisfied By Component(s)	Associated Data Structures
FR-1	The host can create a new room	createHostThread(), generateUniqueRoomID()	roomParticipantMap, allPolls
FR-2	Participants can join a room via a room code	join-hello handler, createParticipantThread()	roomParticipantMap, allVotes
FR-3	The host can create new polls	create-poll handler, Poll struct	allPolls, Poll
FR-4	Participants can vote and predict on polls	vote handler	UserVote, allVotes
FR-5	The host can reveal poll results	reveal-poll handler	Poll.revealed
FR-6	The system must auto-expire rooms after host leaves	createRoomExpiryThread()	expiringRooms, roomParticipantMap
FR-7	Communication must be real-time over WebSockets	onMessage(), dispatcherThreadFunction(), Communication.svelte.ts	Message, ConnectionThreadInfo
FR-8	Host cannot vote or predict	vote handler (host check logic)	allConnectionThreads, ConnectionThreadInfo
FR-9	Participants cannot create or reveal polls	create-poll, reveal-poll handlers (role checks)	ConnectionThreadInfo

8.0 STATEMENT OF CONTRIBUTION

Carmel Kurland

- Head of UI. Led the frontend design and implementation using Svelte and SvelteKit. Created the majority of the UI/UX, including animations, styling, page transitions, and component interactions.

Typo Crawley

- Contributed to the frontend. Assisted with user interface components and layout styling. Helped test visual consistency and user experience across devices.

Xristopher Aliferis

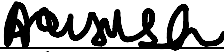
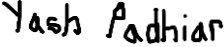
- Lead backend developer. Designed and implemented the entire server architecture in C++. Built the WebSocket communication system, poll management logic, threading model, and overall infrastructure.

Aayush Bhanot

- Supported backend development. Helped debug server-side concurrency issues and assisted in implementing key message handlers for poll voting and room validation.

Yash Padhiar

- Supported backend development. Contributed to server-side testing, validation logic, and implementation of utility functions like poll deletion and room cleanup.

Name	Signature	Signature Date
Carmel Kurland		2025-04-11 7:56:56 PM
Tygo Crawley		2025-04-11 8:38:27 PM
Xristopher Aliferis		2025-04-10 12:51:56 PM
Aayush Bhanot		2025-04-10 11:26:22 PM
Yash Padhiar		2025-04-11 10:05:04 PM